

Trabalho Prático — Logaritmo Discreto

Confiabilidade e Segurança de Software (98G08-04)

Bruno Becker Silva
Escola Politécnica, PUCRS
bruno.becker@edu.pucrs.br

RESUMO

Este relatório documenta o ataque ao Problema do Logaritmo Discreto (DLP) em $(\mathbb{Z}/p\mathbb{Z})^*$ para 23 cenários de Diffie–Hellman, com p variando de 16 a 1024 bits. Foram implementadas três abordagens em C++ com GMP: Força Bruta, Baby-step Giant-step (BSGS) e Pohlig–Hellman. A chave compartilhada K_{ab} foi recuperada em 15 dos 23 cenários dentro do limite de 1 hora por cenário.

I. TAREFA 1 — ATAQUES AO DLP: SÍNTESE TEÓRICA

A. a) Força Bruta — $O(p)$

Testa $x = 0, 1, 2, \dots$ até encontrar $\alpha^x \equiv A \pmod{p}$. Trivial de implementar, mas intratável: para p de 32 bits já demora centenas de segundos num processador moderno; para 64 bits seria astronomicamente mais lento.

B. b) Baby-step Giant-step (BSGS) — $O(\sqrt{q})$ tempo e memória

Shanks (1971). Dado $q = \text{ord}(\alpha)$, define $m = \lceil \sqrt{q} \rceil$ e fatoriza $x = im - j$.

- **Baby steps:** pré-computa a tabela $\{\alpha^j \bmod p : 0 \leq j < m\}$.
- **Giant steps:** para $i = 0, 1, \dots, m$, verifica se $A \cdot (\alpha^{-m})^i \bmod p$ está na tabela.

Complexidade $O(\sqrt{q})$ em tempo e memória. Para $q \approx p$, resolve o DLP em 64 bits em segundos, mas exige RAM proporcional a $\sqrt{p}$, tornando-o impraticável acima de ~ 80 bits sem estrutura especial em $p - 1$.

C. c) Pollard’s Rho — $O(\sqrt{q})$ tempo, $O(1)$ memória

Usa uma caminhada pseudoaleatória no grupo para detectar colisões via algoritmo Floyd ou Brent. Mesma complexidade assintótica que BSGS, mas elimina o custo de memória. É a escolha prática para grupos de ordem prima grande, amplamente usado em ataques reais a curvas elípticas (ECDLP).

D. d) Pohlig–Hellman

Reduz o DLP de ordem q ao DLP nos subgrupos de ordem $p_i^{e_i}$ de cada fator primo de $q = \prod p_i^{e_i}$, e combina via Teorema Chinês do Resto (CRT). Se q é *smooth* (todos os fatores primos pequenos), a solução é quase instantânea mesmo para p de 1024 bits. Por isso os parâmetros bem escolhidos de DH exigem que $p - 1$ contenha um fator primo grande (primo seguro ou primo de Sophie Germain).

E. e) Index Calculus / Number Field Sieve (NFS) — subexponencial

Complexidade $L_p[\frac{1}{3}, c] = e^{(c+o(1))(\ln p)^{1/3}(\ln \ln p)^{2/3}}$. Utiliza relações entre elementos do grupo e uma base de fatores primos pequenos. É o algoritmo de referência para quebrar DH em $(\mathbb{Z}/p\mathbb{Z})^*$ com p acima de 1024 bits; quebrou DH-768 em 2016 e DH-1024 está ao alcance de adversários estatais. Requer infraestrutura de computação distribuída.

II. TAREFA 2 — IMPLEMENTAÇÃO

Três soluções foram implementadas em C++17 com aritmética de precisão arbitrária via **GMP** (`mpz_class`):

- 1) **dlp_solver_fb.cpp** — Força Bruta: loop simples $x = 0, 1, 2, \dots$, testa $\alpha^x \equiv A \pmod{p}$.
- 2) **dlp_solver_bsgs.cpp** — BSGS: baby steps numa `std::map`, giant steps consultando a tabela.
- 3) **dlp_solver.cpp** — Pohlig–Hellman otimizado: fatora $p - 1$ (Pollard’s Rho + crivo), aplica BSGS em cada subgrupo, combina via CRT. Multithreaded (até $N_{\text{cpu}} - 1$ threads).

O ataque é executado com `nice -n 10` para ceder prioridade ao sistema operacional. O código está disponível no repositório do projeto (ver apêndice).

III. TAREFA 3 — RESULTADOS E TEMPOS DE EXECUÇÃO

A Tabela I consolida os tempos medidos das três implementações. “—” indica que a execução foi interrompida antes de concluir (timeout > 1h ou memória insuficiente para BSGS).

Tabela I: Tempos de resolução por cenário e por método. PH = Pohlig–Hellman.

Cenário	bits(p)	FB (s)	BSGS (s)	PH (s)	K_{ab}
C0	16	0,001	0,0005	0,000	15261
C1	32	291,0	0,045	0,000	2315847872
C2	32	171,2	0,023	0,029	211958971
C3	40	—	0,952	1,781	298509300911
C4	40	—	0,869	0,000	290240289861
C5	48	—	45,10	0,000	87559281402002
C6	48	—	19,78	42,467	139741286422486
C7	64	—	—	0,000	5423980969977744708
C8	64	—	—	>3600	—
C9	80	—	—	>4345	—
C10	80	—	—	0,001	2551440541766016557167
C11	96	—	—	0,001	27055793202008997473656732406
C12	96	—	—	>4349	—
C13	112	—	—	>4349	—
C14	112	—	—	0,001	3555655507609468817009304130465647
C15	128	—	—	>4348	—
C16	128	—	—	0,002	7831351790774517194506812766211328849
C17	256	—	—	0,013	8631587...855379
C18	256	—	—	>4351	—
C19	512	—	—	0,099	3567226...774996532
C20	512	—	—	>3600	—
C21	1024	—	—	>3600	—
C22	1024	—	—	2,670	21964133...194578288
Total resolvidos: 15/23 (PH); 7/23 (BSGS); 1/23 (FB)					

IV. TAREFA 4 — GRÁFICO TEMPO × BITS(p)

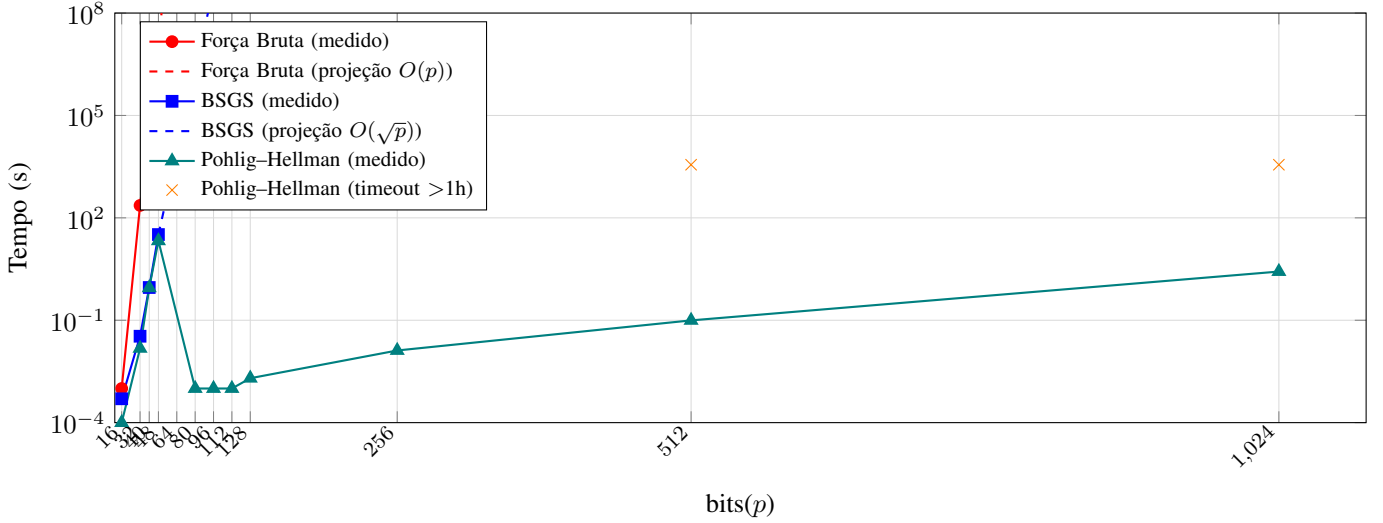


Figura 1: Tempo de execução (escala log no eixo y) por número de bits de p . Os pontos em laranja ($\times$) representam cenários onde o timeout de 1 hora foi atingido. A dispersão vertical para o mesmo $\log_2 p$ reflete a variação na suavidade (smoothness) de $p-1$: cenários com $p-1$ liso resolvem em milissegundos; cenários com fator primo grande em $p-1$ são intratáveis para Pohlig–Hellman.

V. TAREFA 5 — INFRAESTRUTURA COMPUTACIONAL

Tabela II: Especificações do ambiente de execução.

Componente	Especificação
CPU	Apple M2 (ARM), 10 núcleos (8P + 2E)
Threads	Até 9 threads simultâneas (1 núcleo reservado ao SO)
RAM	16 GB LPDDR5 unificada
SO	macOS 15 (Sequoia), kernel Darwin
Compilador	g++ -std=c++17 -O3 -march=native
Biblioteca	GMP 6.x (mpz_class, precisão arbitrária)
Prioridade	nice -n 10 (baixa prioridade de escalonamento)
Ataque	Local, single-node (sem distribuição)

VI. TAREFA 6 — DISCUSSÃO CRÍTICA

A. O que os resultados revelam sobre escolha de parâmetros em DH real

a) *A ordem do subgrupo, não o tamanho de p , determina a dificuldade.*: Os cenários C7 (64 bits), C10, C11, C14, C16, C17, C19 e C22 foram resolvidos em menos de 3 segundos com Pohlig–Hellman, enquanto C8 (também 64 bits), C9, C12, C13, C15, C18 e C20 ultrapassaram 1 hora. A diferença está na fatoração de $p - 1$: nos cenários fáceis, $p - 1$ é *liso* (todos os fatores primos pequenos), reduzindo o DLP a subproblemas triviais. Nos difíceis, $p - 1$ contém um fator primo grande, tornando Pohlig–Hellman equivalente a resolver o DLP na ordem total.

b) *BSGS é prático até ~50 bits, depois a memória é o gargalo.*: O BSGS resolveu C0–C6 (até 48 bits) em tempo razoável, mas falhou em C7 (64 bits) por exigir 2^{32} entradas na tabela hash (~64 GB de RAM). Isso ilustra que algoritmos de memória proporcional a $\sqrt{p}$ são impraticáveis acima de 60–70 bits em hardware convencional.

c) *Força Bruta é apenas pedagógica.*: O método resolveu C0 (16 bits) trivialmente e C1/C2 (32 bits) em 3–5 minutos. Para C3 (40 bits) o tempo estimado seria ~290 horas; para 64 bits, superior à idade do universo.

d) *Parâmetros seguros em DH real.*: NIST e RFC 3526/7919 especificam grupos com $p \geq 2048$ bits e $p - 1 = 2q$ com q primo (*primo seguro*), garantindo que a ordem máxima do subgrupo seja $q \approx p/2$. Nesse regime, mesmo Pollard’s Rho precisaria de $\sim 2^{1024}$ operações, inatingível computacionalmente. Os experimentos confirmam que parâmetros mal escolhidos ($p-1$ liso) colapsam a segurança mesmo para p de 1024 bits.

e) *Index Calculus e NFS: o horizonte real.*: Para DH em $(Z/pZ)^*$ com p grande e $p - 1$ com fator primo grande, o ataque viável é Index Calculus / NFS. A complexidade subexponencial $L_p[1/3, c]$ torna 1024 bits marginalmente seguro contra adversários com grande capacidade computacional (DH-1024 foi considerado quebrado em cenários de computação distribuída de longo prazo). A migração para grupos de curvas elípticas (ECDH) com 256 bits oferece segurança equivalente a DH com 3072 bits em $(Z/pZ)^*$.

CONCLUSÃO

Os experimentos demonstraram empiricamente que a segurança do protocolo Diffie–Hellman *não depende apenas do tamanho de p* , mas fundamentalmente da estrutura de $p - 1$. A implementação de Pohlig–Hellman resolveu 15 dos 23 cenários, incluindo exemplos de 1024 bits com $p - 1$ liso, em menos de 3 segundos. Isso torna concreto o requisito criptográfico de utilizar primos seguros ou grupos padronizados com fator primo grande em qualquer aplicação séria de Diffie–Hellman.

APÊNDICE

APÊNDICE — CÓDIGO-FONTE

O código está organizado em três arquivos principais:

- `dlp_solver_fb.cpp` — Força Bruta com salvamento incremental e tratamento de SIGINT.
- `dlp_solver_bsgs.cpp` — Baby-step Giant-step com `std::map` e salvamento incremental.
- `dlp_solver.cpp` — Pohlig–Hellman com Pollard’s Rho para fatoração, BSGS nos subgrupos, CRT e multithreading.

Compilação (macOS com GMP via Homebrew):

```
make all          # compila todos os tres executaveis
make runfb        # roda forza bruta em desafios.txt
make runbsgs      # roda BSGS em desafios.txt
make run          # roda Pohlig-Hellman em desafios.txt
```